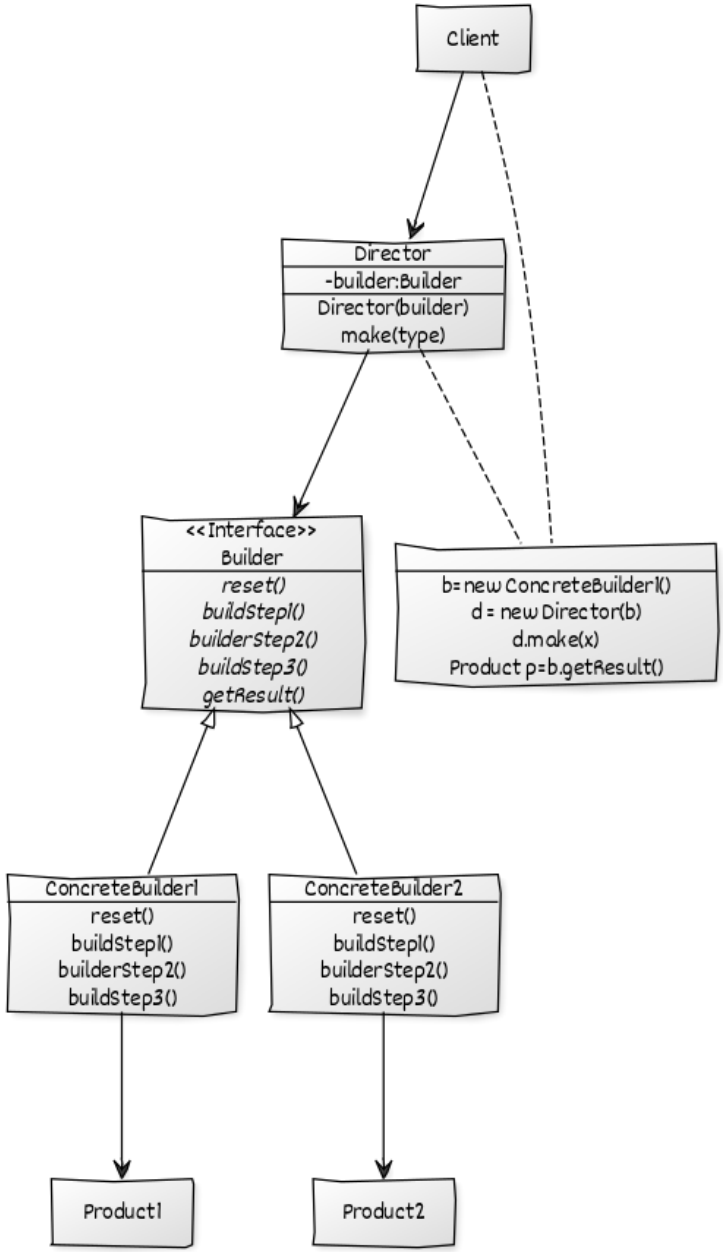
Builder

- Creational Pattern
- Split the responsibility of constructing a complex object from its representation. The knowledge of creating an instance of a complex class is outside of that class.
- Analogy: a civil engineer knows how to build a bridge, not the bridge itself.
- Easy to extend to other types of the bridges (Salgina, Golgen Gate, Tower bridge, Rialto,…)
- In reality Builder and director are often merged



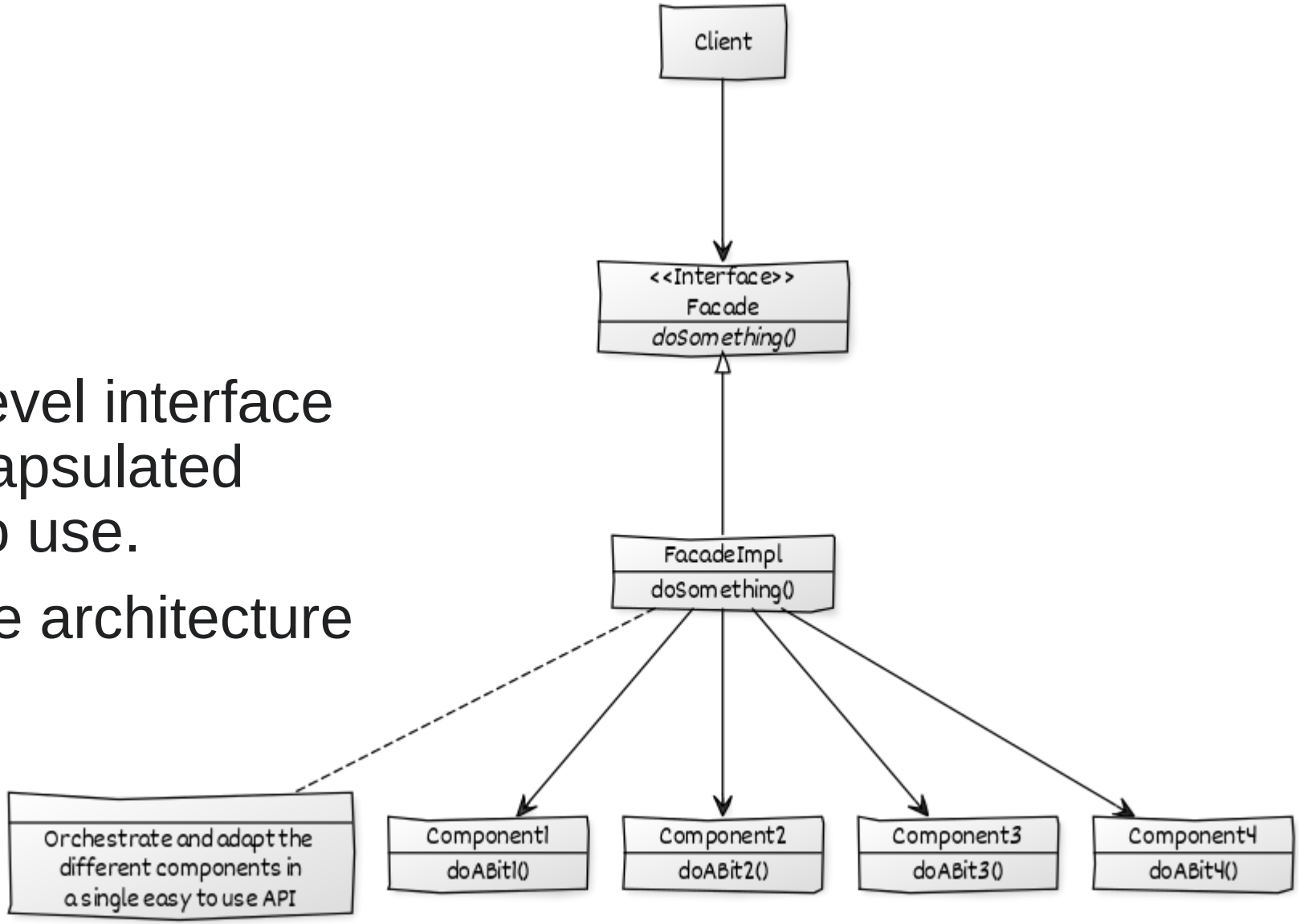
Builder

```
class Car {
	int rimSize;
	int nbOfDoors;
	int engineSize;

	public Car(int rimSize, int nbOfDoors, int
	engineSize) {
		super();
		this.rimSize = rimSize;
		this.nbOfDoors = nbOfDoors;
		this.engineSize = engineSize;
	}

	public int getRimSize() {
		return rimSize;
	}

	public int getNbOfDoors() {
		return nbOfDoors;
	}

	public int getEngineSize() {
		return engineSize;
	}
}

interface CarBuilder {
	CarBuilder setRimSize(int size);
	CarBuilder setDoors(int nbOfDoors);
	CarBuilder addEngine(int cubicSize);
	CarBuilder addSteeringWheel();
	Car build();
}
```

```
class BasicCarBuilder implements CarBuilder {
	int rimSize;
	int nbOfDoors;
	int engineSize;

	public CarBuilder setRimSize(int size) {
		this.rimSize = size;
		return this;
	}

	public CarBuilder setDoors(int nbOfDoors) {
		this.nbOfDoors = nbOfDoors;
		return this;
	}

	public CarBuilder addEngine(int cubicSize) {
		this.engineSize = cubicSize;
		return this;
	}

	public CarBuilder addSteeringWheel() {
		return this;
	}

	public Car build() {
		return new Car(this.rimSize, this.nbOfDoors,
		this.engineSize);
		}
	}

	public static void main(String[] args) {
		Car c = new BuilderTest().new
		BasicCarBuilder().addEngine(6000).addSteeringWheel().s
		etRimSize(21).setDoors(2)
		.build();
	}
}
```

Facade

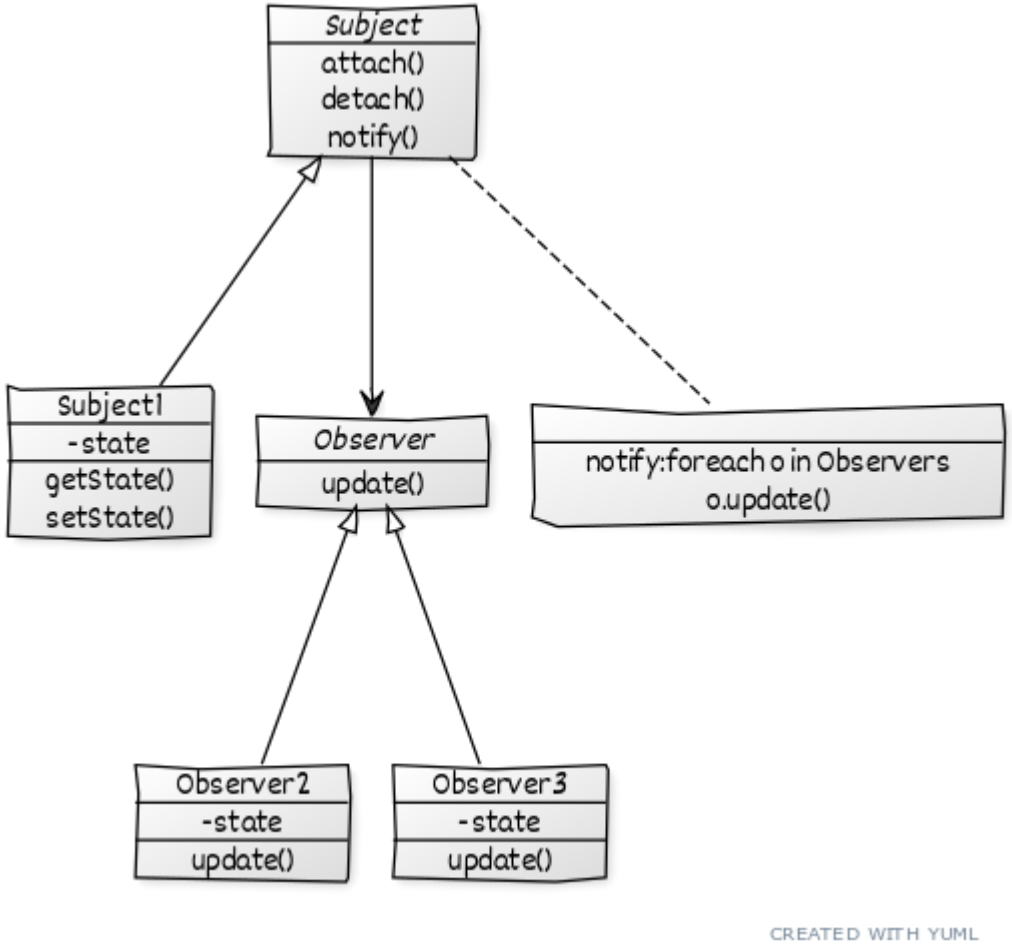
- Structural Pattern
- Provides a higher-level interface that makes the encapsulated subsystem easier to use.
- Similar to the façade architecture pattern



Facade

Observer

- Behavioral pattern
- Similar to the publish/subscribe architecture pattern
- Pushes notification of changes to a set of interested parties



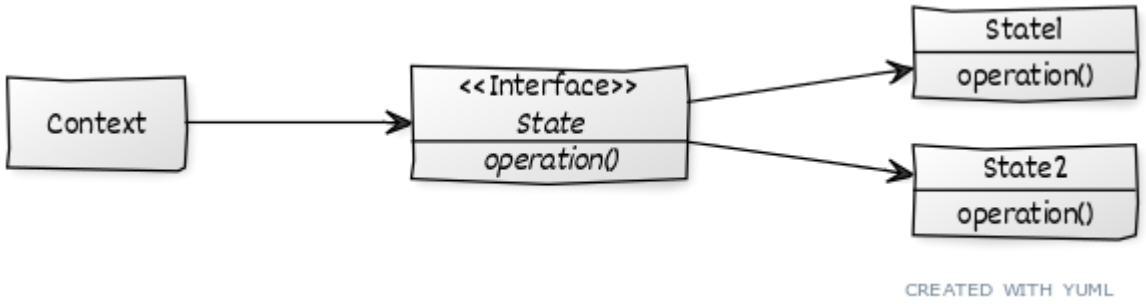
Observer

```java
static class Pair<T, K> {
	private T t;
	private K k;

	Pair(T t, K k) {
		this.t = t;
		this.k = k;
	}

	public T getT() {
		return t;
	}

	public K getK() {
		return k;
	}

	public String toString() {
		return "<" + t + "," + k + ">";
		}
	}

}
```

```java
static class ElectionUpdate {
	Map<String, Long> results = new HashMap<>();

	public ElectionUpdate(Pair<String, Long>... scores) {
	Arrays.stream(scores).map(s -> this.results.put(s.getT(), s.getK()));
	}

	public String toString() {
	return "Election Update : " + results.keySet().stream().map(c -> (c + "=" +
	results.get(c).toString()))
	.collect(Collectors.joining(", ", "{", "}"));
	}
}

static abstract class AbstractElectionUpdateSubsriber implements
Subscriber<ElectionUpdate> {
	private Subscription subscription;

	public void onSubscribe(Subscription subscription) {
		this.subscription = subscription;
		this.subscription.request(1);
	}

	public void onNext(ElectionUpdate item) {
		System.out.println(this.getClass() + " : " + item);
		 this.subscription.request(1);
	}

	public void onError(Throwable throwable) {
		System.out.println(AbstractElectionUpdateSubsriber.class + " : " +
		throwable.toString());
		this.subscription.cancel();
	}

	public void onComplete() {
		System.out.println("Done");
	}
}
```

State

- Behavioral pattern
- Implements the concept of state machine
- An object changes its behavior when the internal state changes
- Adding new state do not change existing states.



State

```java
package ch.unige.se;

public class StateTest {

static class TrafficLight {
	private TrafficLightState state = new RedState();

	public void nextState() {
	state.next(this);
	printStatus();
	}

	public void printStatus() {
	state.printStatus();
	}

	public void setState(TrafficLightState state) {
	this.state = state;
	}
}

static interface TrafficLightState {
	void next(TrafficLight trf);
	void printStatus();
}

static class RedState implements TrafficLightState {
	public void next(TrafficLight trf) {
		trf.setState(new OrangeState());
	}
	public void printStatus() {
		System.out.println("Red");
	}
}

static class OrangeState implements TrafficLightState {
	public void next(TrafficLight trf) {
		trf.setState(new GreenState());
	}
	public void printStatus() {
		System.out.println("Orange");
	}
}

static class GreenState implements TrafficLightState {
	public void next(TrafficLight trf) {
		trf.setState(new RedState());
	}
	public void printStatus() {
		System.out.println("Green");
	}
}

public static void main(String[] args) {
	TrafficLight light = new TrafficLight();
	light.printStatus();
	light.nextState();
	light.nextState();
	light.nextState();
	light.nextState();
	}
}
```